Relatório: Servidor UDP com Entrega Ordenada

Sistemas Distribuídos (SD)

1 Introdução e Conceitos Base

1.1 Receção vs. Entrega à Aplicação

No contexto do protocolo UDP, existe uma distinção fundamental entre uma mensagem ser *recebida* e ser *entregue à aplicação*:

- **Recebida (pelo Sistema Operativo):** O pacote chega à placa de rede. O SO processa-o e coloca-o num *buffer* (fila de espera) associado ao *socket*. A mensagem já está no computador, mas o programa Java ainda não a processou.
- **Entregue à aplicação:** Ocorre quando o código executa a função `socket.receive()`. Nesse instante, o SO retira a mensagem do buffer e entrega-a ao programa.

1.2 O Problema de Descartar Mensagens Fora de Ordem

Como a rede pode entregar pacotes fora de ordem, a aplicação pode receber uma mensagem adiantada (ex: mensagem $N = 3$ quando espera $N = 2$). Se a aplicação simplesmente *descartar* a mensagem $N = 3$ e pedir a retransmissão da $N = 2$, quando a $N = 2$ chegar, a aplicação vai precisar novamente da $N = 3$. Isto obriga o cliente a retransmitir uma mensagem que já tinha chegado com sucesso, causando **retransmissões desnecessárias** e desperdício de largura de banda.

1.3 A Solução: Buffer Temporário e Entrega em Cascata

A solução recomendada é guardar temporariamente as mensagens fora de ordem numa estrutura (ex: `Map<Integer, String>`). Quando a mensagem em falta finalmente chegar, o servidor processa-a e, de seguida, processa em cascata todas as mensagens que já tinha guardadas, evitando retransmissões.

2 Arquitetura da Solução

2.1 Estruturas de Dados

Foram definidas duas estruturas principais (campos estáticos da classe, para permitir o acesso pelo método exigido):

- `List<String> finalMessages`: Lista de receção (mensagens já entregues por ordem).
- `Map<Integer, String> outOfOrderMessages`: Estrutura temporária (mensagens que chegaram fora de ordem).

2.2 O Método `processDeliveredMessages`

Conforme exigido na ficha, foi criado o método:

```
public static int processDeliveredMessages(int nLastMessageInOrder,
int nCurrentMessage,
String currentMessage)
```

Este método:

1. Se $N == L + 1$, entrega a mensagem, incrementa L e verifica no `Map` se a mensagem $L + 1$ já está guardada, entregando-a em cascata.
2. Se $N > L + 1$, guarda a mensagem no `Map` (se ainda não existir, ignorando duplicados na temporária) e devolve L inalterado.
3. Se $N \leq L$, ignora a mensagem (duplicado já entregue).

3 Regras de Resposta e Duplicados

3.1 Contrato de Resposta

A ficha define que a resposta ao cliente deve ser:

- **Echo do datagrama recebido** se a mensagem foi entregue com sucesso neste passo.
- `waitingfor,<L+1>` se a mensagem **não** foi entregue neste passo.

Para cumprir o contrato literalmente, a condição usada no `main` é `if (L != oldL)`. Como o método só altera L quando há pelo menos uma entrega, `L != oldL` significa que houve entrega (e vice-versa).

3.2 Tratamento de Duplicados

- **Duplicado já entregue** ($N \leq L$): É ignorado. Como não é entregue neste passo, a resposta é `waitingfor,<L+1>`.
- **Duplicado na estrutura temporária** ($N > L + 1$ e já está no `Map`): É ignorado. A resposta também é `waitingfor,<L+1>`.

Nota: Embora responder `ok,<L>` fosse mais amigável para o cliente, isso desviar-se-ia do contrato explícito da ficha.

4 Teste e Validação

A sequência de teste proposta é: `1, 3, 4, 2, 3`. O estado inicial é $L = 0$, `Map` vazio e `finalMessages` vazia.

4.1 Confirmações do Teste

- **Na receção de 2:** São entregues 2, 3 e 4 (efeito de cascata). O `Map` fica vazio.
- **Na receção do último 3:** L não muda ($L = 4$ antes e depois). Não há mensagens entregues neste passo (`deliveredThisStep` vazio). A resposta é `waitingfor,5`, pois $L = 4$.

Passo	Recebido	Resposta	L após	Map após	Entregues no passo
1	`1, alpha`	`1, alpha` (echo)	1	`{}`	`[alpha]`
2	`3, gamma`	`waitingfor,2`	1	`{3=gamma}`	`[]`
3	`4, delta`	`waitingfor,2`	1	`{3=gamma, 4=delta}`	`[]`
4	`2, beta`	`2, beta` (echo)	4	`{}`	`[beta, gamma, delta]`
5	`3, gamma`	`waitingfor,5`	4	`{}`	`[]`

Tabela 1: Registo de teste da sequência 1, 3, 4, 2, 3.

5 Limitações e Reflexão Crítica (CA5)

1. **Estado Estático Global:** O L, a `finalMessages` e o `outOfOrderMessages` são partilhados por todas as invocações (campos estáticos). Isto limita o servidor a um único cliente com uma única sequência de numeração. Para suportar múltiplos clientes, seria necessário um `Map<EndereçoCliente, EstadoCliente>`, onde cada cliente tem o seu próprio L e o seu próprio `Map`.

2. **Falta de Timeout:** Se a mensagem em falta nunca chegar, o `Map` cresce indefinidamente. O ideal é implementar um temporizador (timeout) que limpe a estrutura temporária após X segundos, assumindo que a mensagem se perdeu.

3. **Limite de Crescimento:** Um cliente malicioso (ou com um bug) pode enviar uma mensagem $N = 1\,000\,000$, fazendo o servidor alocar memória desnecessária. Seria necessário impor um limite máximo para o número de mensagens guardadas no `Map`.

4. **Duplicados e Respostas:** Optou-se por responder `waitingfor` a duplicados para seguir literalmente o contrato da ficha. No entanto, numa aplicação real, responder com um ACK (ex: `ok,<L>`) a duplicados já entregues ajudaria o cliente a perceber que não precisa de retransmitir.